Government College of Engineering, Jalgaon **(An Autonomous Institute of Government of Maharashtra)**	
Name :	**PRN :**
Class : L. Y. B.Tech Computer	**Batch:**
Subject : CO456U Cloud Computing Lab	**Sem :** VIIIth
Course Teacher: Prof. S. D. Cheke	**Academic Year :** 2025-26
Date of Performance :	**Date of Completion :**

Practical No - 6

Aim: Write a docker file to containerize the python flask application

Theory :

Objective: To create a Dockerfile for containerizing a Python Flask application and run it inside a Docker container.

What is Docker?

Docker is a platform used to develop, ship, and run applications inside lightweight containers.

- Ensures consistency across environments
- Faster deployment compared to virtual machines

What is Containerization?

Containerization is the process of packaging an application along with its dependencies so it can run anywhere.

What is Flask?

Flask is a lightweight web framework used to build web applications in Python.

What is a Dockerfile?

A **Dockerfile** is a text file that contains instructions to automatically build a Docker image.

- Handles multiple users efficiently
- Used as web server, reverse proxy, and load balancer

Requirements

- Docker installed
- Python Flask application
- Internet connection

Sample Flask Application

Create a file **app.py**:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello, Docker Flask App!"

if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000)
```

Create **requirements.txt**:

```
flask
```

Create a file named **Dockerfile**:

```
# Use official Python image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy files
COPY . /app

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```

Steps to Build and Run Container

Step 1: Build Docker Image

```
docker build -t flask-app .
```

Step 2: Run Docker Container

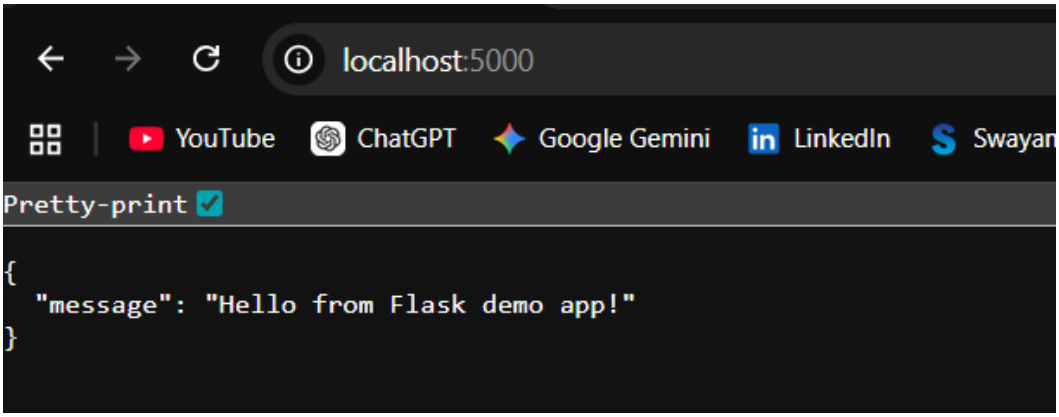
```
docker run -p 5000:5000 flask-app
```

Step 3: Open Browser

http://localhost:5000

```
Windows PowerShell
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\Me\College\CCL\flask_app> docker build -t flask-demo .
[+] Building 19.2s (10/10) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                  0.1s
 => => transferring dockerfile: 269B                                                                                  0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                                   5.0s
 => [internal] load .dockerignore                                                                                     0.1s
 => => transferring context: 134B                                                                                     0.0s
 => [internal] load build context                                                                                     0.2s
 => => transferring context: 524B                                                                                     0.0s
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:804ddf3251a60bbf9c92e73b7566c40428d54d0e79d3428194edf40d     5.9s
 => => resolve docker.io/library/python:3.12-slim@sha256:804ddf3251a60bbf9c92e73b7566c40428d54d0e79d3428194edf40d     0.1s
 => => sha256:5435b2dcdf5cb7faa0d5b1d4d54be2c72a776fab9a605336f5067d6e9ecb5976 29.78MB / 29.78MB                      3.4s
 => => sha256:f0bdb572205ec70132c0fbc66a335ee5fd1614c08d686e8e921a9f55a6c38b52 250B / 250B                            1.4s
 => => sha256:d4c207a1ca273594af4c026252870b4b165d536b3cb53b1246d6805e4e6b34b2 12.11MB / 12.11MB                      5.0s
 => => sha256:25981ed25cff34d7d714ea438c95731e427ea9827b771ed102d6e0fc30eeabe2 1.29MB / 1.29MB                        3.1s
 => => extracting sha256:5435b2dcdf5cb7faa0d5b1d4d54be2c72a776fab9a605336f5067d6e9ecb5976                             0.9s
 => => extracting sha256:25981ed25cff34d7d714ea438c95731e427ea9827b771ed102d6e0fc30eeabe2                             0.2s
 => => extracting sha256:d4c207a1ca273594af4c026252870b4b165d536b3cb53b1246d6805e4e6b34b2                             0.4s
 => => extracting sha256:f0bdb572205ec70132c0fbc66a335ee5fd1614c08d686e8e921a9f55a6c38b52                             0.1s
 => [2/5] WORKDIR /app                                                                                                0.2s
 => [3/5] COPY requirements.txt .                                                                                     0.1s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                          5.9s
 => [5/5] COPY . .                                                                                                    0.2s
 => exporting to image                                                                                                1.4s
 => => exporting layers                                                                                               0.8s
 => => exporting manifest sha256:5f97db37934dac6c4755dfee8ee2da4805e39d8465db0d78549b035449d2d62f                      0.0s
 => => exporting config sha256:ea47cf5ccd63f0e61034a6d5f96f334b420229c1780b5e297d8b3602ee1a989d                       0.0s
 => => exporting attestation manifest sha256:851ec967c20ea22ce4227eb785c3b75c133d6bf1198054139674dff5e7f09752         0.1s
 => => exporting manifest list sha256:5409ca9b3c4f7f5143d833d17b313bbeab9e9209b08089d529c225a0b56a0101                0.0s
 => => naming to docker.io/library/flask-demo:latest                                                                  0.0s
 => => unpacking to docker.io/library/flask-demo:latest                                                               0.2s
PS D:\Me\College\CCL\flask_app> docker run --rm -p 5000:5000 flask-demo
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
```



Successfully created a Dockerfile, built a Docker image, and ran a Python Flask application inside a container.

Conclusion: In this way, we have successfully implemented and completed the containerization of a Python Flask application using Docker.

Prof. S. D. Cheke
Course Teacher